CmpSc 463 - Design Analysis Algorithms

Project 2 - Evacuation Route Planner

Jonathan Alavez Reyes & Owen Mallon

Due: 4/26/2026

Github Repository: https://github.com/AlrJohn/cs463_Final_Project.git

Link to app: Evacuation Route and Priority Planner

Link to Video Demonstration: Video Demonstration

Description & Significance:

In this modern era, conflict and war are present all around the world, impacting the lives of everyone in and around the zones of conflict. While this program and project do not aim to solve these conflicts, we aimed to develop a program that would be beneficial and useful within these zones as a way to help any and all affected populations. While conflicts have a long list of cause and effect events, one undeniable fact is the impact it brings to the lives of everyday civilians living both in and around the conflict. The UN reports that civilian deaths within the conflict zone have been rising alarmingly in the past few years, with just recently a 40% increase in 2024 compared to 2023, with numbers appearing to once again rise by the end of this year. This sole fact is what led us to develop our project, an Evacuation Route Planning Simulator.

When trying to figure out ways to help civilian populations get both out of harm's way and to safety and health professionals, one of the most difficult things is attempting to find a safe path to travel. With how rapidly and ever-changing conflicts may be, our program operates using as close to real-time data as possible, obtained by allowing users to update the map based on the current situation, and then calculating the fastest way to a specific location in dangerous situations. While there are many different things happening during the time period following an attack, the most important step first is focusing on those who need medical aid and getting people to a shelter to start. These primary goals are exactly what our project aimed to assist with.

Through the collection of algorithms developed over the course of this project, we aimed to solve 2 primary tasks within conflict zones. Not only did we look to decide which group in a zone should be picked up next, but also the safest route from their location to a shelter or hospital, based on their needs. Using a modified explicit Dijkstra-style implementation (min-heap) for weighted route computation, we were able to solve both these tasks through the

development of our own Streamlit application with a python front and backend. The MVP features within our application are:

- fixed scenario based on a simplified ***Al-Mafraq, Jordan*** setting
- one vehicle, four pickup groups, two shelters, and one hospital
- user-controlled road updates: open, dangerous, blocked
- explicit Dijkstra-style implementation (min-heap) for weighted route computation
- A greedy prioritization rule for selecting the next pickup group
- shelter and hospital capacity checks
- one-page Streamlit UI with map, controls, outputs, and ranking table

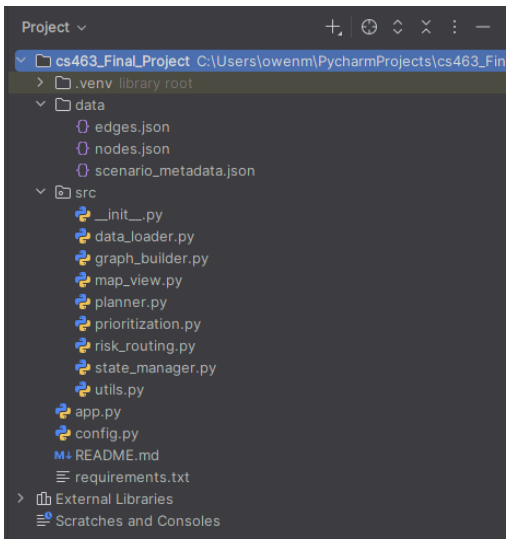
Code Structure & Description:

The general structure of our code is as follows:

```
app.py
config.py
requirements.txt
README.md

data/
  nodes.json
  edges.json
  scenario_metadata.json

src/
  data_loader.py
  graph_builder.py
  risk_routing.py
  prioritization.py
  planner.py
  map_view.py
  state_manager.py
  utils.py
```



We have the 4 primary files that users interact with at the top that act as the front-end facing portion that users will be able to interact with and access. We then have the two important

data and source code folders that house the backend of the application. Of the 4 primary files, users interact with app.py the most, as it is the program that launches our Streamlit application, while the remaining 3 are configuration settings, dependency download requirements, and lastly a text file containing a description and steps to run the program successfully.

Within the data folder, we have JSON files storing the graph nodes, edges, and scenario_metadata. The node file contains key locations on the map, such as the current vehicle location, pick up groups, hospitals, and shelters. Each location contains unique information essential to its location on the map, as well as information used within the algorithms, such as severity level, capacity, and more. Within the edge file, we have all unique edges that exist within the graph, all identifiable by a unique edgeID. In addition, each edge contains information regarding the nodes they are attached to, the direction it leads in, the distance, travel time, danger level, and the danger penalty incurred in case the path is chosen to go through that edge. Since all edges share the same information, the JSON file is organized in the following order: {id, from_node, to_node, distance, travel_time, status, danger_penalty}.

Lastly, the scenario_metadata file contains our toy demonstration data to show explicitly how our program functions, and that it does function. This file, as time goes on, should store all the different maps built using our function, to allow easy switching between the different cities and situations; however, for this case, we only have 1 test situation named "***Al-Mafraq Simplified Evacuation Scenario***" that will be more discussed and displayed within the verification and analysis stages of this report.

The src file contains all of the backend code for our program, such as data loading, graph building, map viewing, and others. All files within this folder are coded using Python and work together to deliver upon the goal and vision for our application.

I. Data_loader.py

This file contains all the functions used to load stored data written in the .json files located within the data folder. In addition to basic reading and loading, this file also contains all the needed functions to convert the data loaded into DataFrames tables, imported from pandas. Each function will return either a list of the loaded data or the pandas dataframe.

II. Graph_builder.py

This file contains all the source code for building the graph map of the city within each scenario. By using all the data stored within the nodes and edges file, the functions within the file concatenate all the data together into the city map used by the routing/planning modules and users. Our graph is built and visualized using NetworkX with node and edge attributes attached, as well as an extra dictionary with node id's as keys for easy lookup in the future.

III. Map_view.py

This module contains all the functions that assist with rendering the map using Folium. The functions within the module draw the node markers and roads, each with unique colors for objects and safety levels, as well as a visualized recommended route overlay. Additionally, this is the file that builds a map legend for users, as well as adding edges and nodes.

IV. planner.py

This module contains the main planning logic for the evacuation support tool. The planning flow is: Find vehicles and waiting pickup groups, test route reachability and

destination feasibility, score each reachable option, and then select the best-ranked plan. This file uses code from the prioritization, routing, and utils files to assist its purpose.

V. prioritization.py

This module contains our greedy scoring and ranking helpers for pickup-group prioritization. This file contains the brain of the selection process, using custom values and formulas to score different options on the map. Using the created score, it can then also rank the reachable options with the agreed project rule priority in the following order: medical-need group first, higher severity first, higher overall score, lower total route cost, and lastly larger group size

VI. Risk_routing.py

This module contains our risk-aware routing utilities for the evacuation planner. It implements the explicit Dijkstra-style shortest path algorithm using a min-heap priority queue. The file is essential to the success of our application, as it is crucial for path-finding and returning users the best possible path from point A to B within the conflict-riddled city.

VII. State_manager.py

This module is responsible for session-state helpers for Streamlit interaction. The code within this file manages road status, the default road snapshot for reset, and the last planner result and ranking list. This file is essential to maintaining and keeping our application front-end loaded, updated, and fresh for users, while saving and storing any created paths.

VIII. utils.py

This module contains all shared helper functions for our lookup, capacity, and output formatting functions. Through functions that calculate remaining capacity, to ones that turn a route into text and format an explanation, this module assists other modules in completing their tasks and ensuring the users get the results they need and understand why.

The core and heart of this project is the shortest-path route planner utilized to give users the best plan of action when travelling around dangerous conflict zones. While each source code file contains some part that leads to this algorithm, it is mostly found within planner, prioritization, and risk routing files.

As previously stated, we use a modified version of Dijkstra's algorithm with a min-heap priority queue. The graph used within the algorithm is a replica of a city, where nodes represent locations and intersections, and edges are the roads between them. Edge weight is calculated as (ROUTE_TRAVEL_TIME_WEIGHT * travel_time) + (ROUTE_DANGER_WEIGHT * danger_penalty). The program begins its decision-making by evaluating the different pickup groups on the map. The evaluation is handled by running Dijkstra's algorithm on each pickup location, giving us a tuple of (path, path cost) for each pickup, using the vehicle's current location as a starting point. For each of the pickups given, we then evaluate based on a few different requirements. Firstly, we test to see if the vehicle can reach the pick-up destination, using Dijkstra's shortest path algorithm once again. If the pickup cannot be reached, it is passed on and added to the evaluations list as such. If reachable, we then determine if it is one of the destination candidates. A destination is considered a candidate if it satisfies the following: if there is a medical need, non-hospital destinations are passed on, if there is no medical need, then

non-shelter destinations are passed on; and if the group_size surpasses the remaining capacity of shelter, then that is passed on. Through these 3 tests, we can determine the best logical location for our vehicle to travel to next, satisfying the needs of the group picked up.

Next, if the pickup is reachable from the current location and there are valid end destination locations, we then decide the actual route. Once again, we use our shortest path algorithm to determine the best path from the pickup spot to the destination. If, for some reason, there is no valid path, we add such to evaluations, stating there is no valid path to any destination within the destination candidates from our pickup spot. However, if given a valid best destination, we then begin to compute all the necessary calculations, such as total route cost, time, danger, and distance. This is all concatenated together and returned to the user in the form of a "route evaluation," giving the user all the information regarding the pickup location, destination, reachability, and pick metrics calculated.

Each pickup is checked, and once completed, we run the final function of our route simulator. The final function is used to determine which path is the best one to take. If no paths exist at the current moment, then we return no path, and the raw evaluations of each pickup state explain why they are not valid paths currently. In the case where valid paths do exist, we then sort based on our custom priority tuple rule and return that path to the user. As stated above, the priority tuple is such:

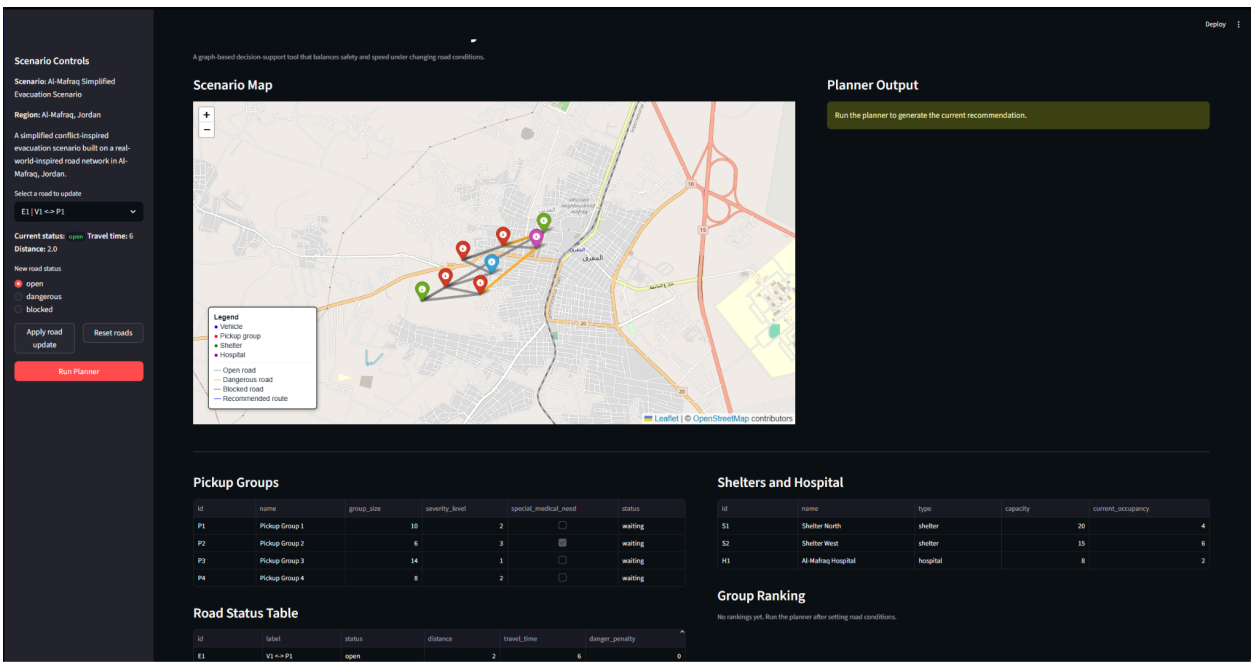
1. medical-need group first
2. higher severity first
3. higher overall score
4. lower total route cost
5. larger group size

Given all this information, our algorithm uses the given knowledge of the current situation to best determine the next best logically possible path of travel from collection to drop-off. This algorithm can be run several times, especially after the previously given path is completed.

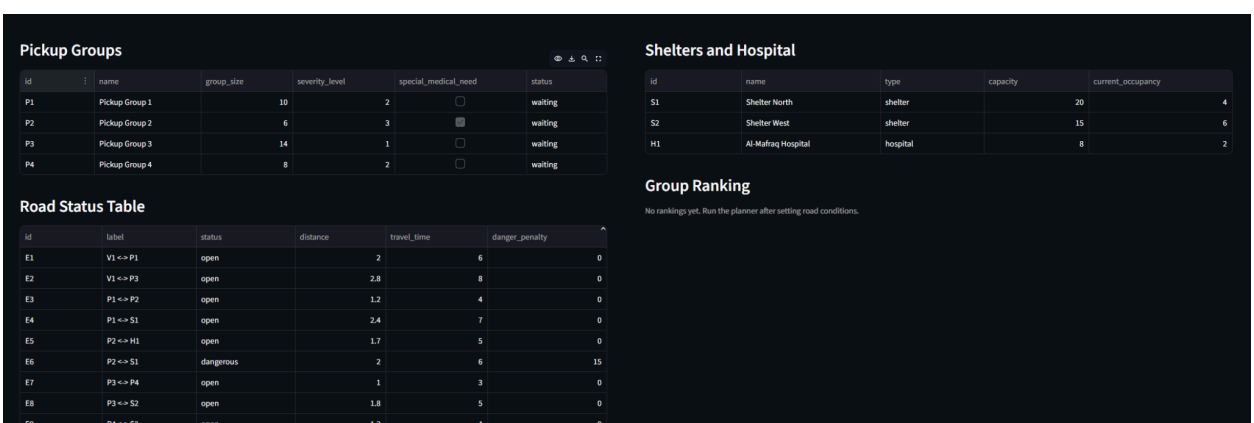
Toy Scenario with Results & Algorithm Verification:

Using our toy-scenario of "***Al-Mafraq Simplified Evacuation Scenario***", this is what the application displays when the route planner application is run:

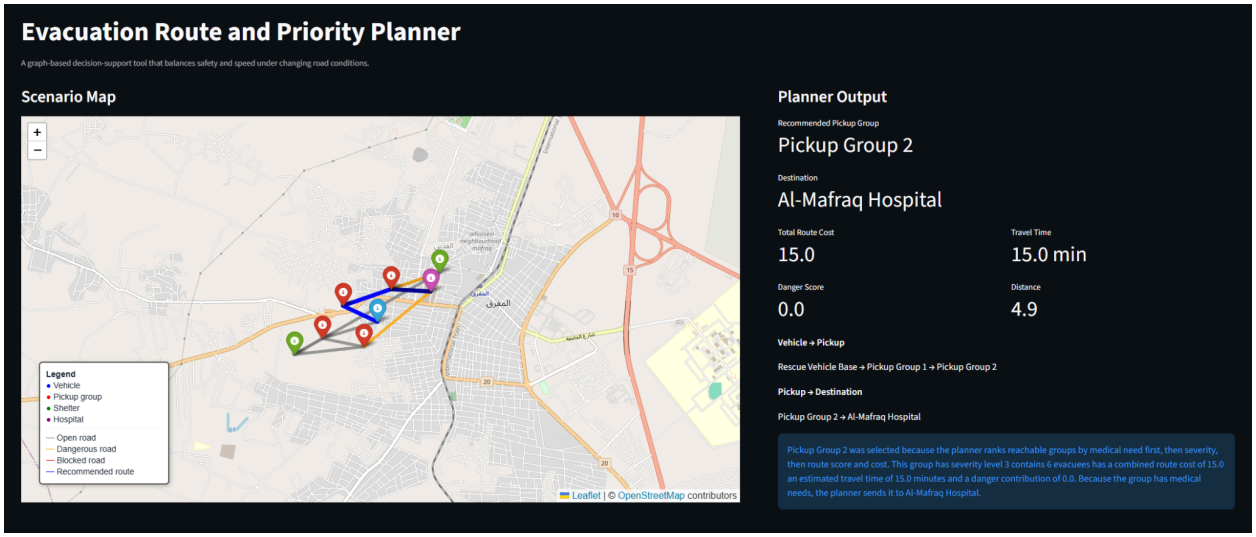
1. Firstly, we have the main screen that users see upon running:

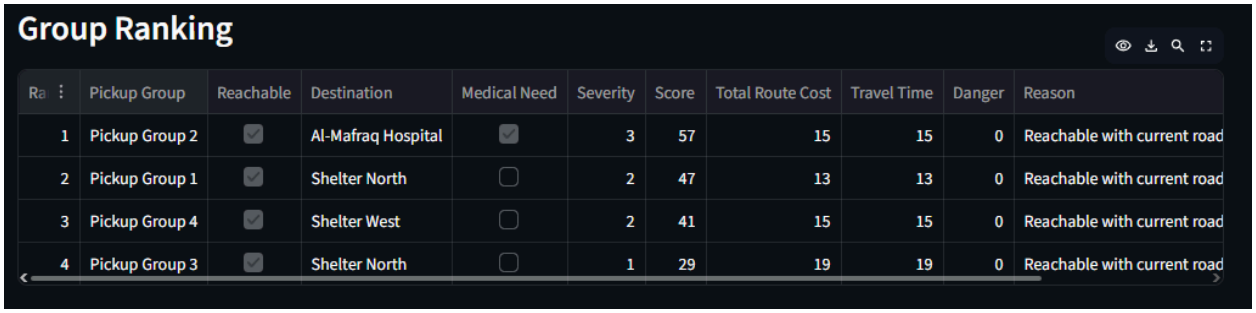


2. Here is our toy example data:



3. Here is what the program outputs when run using the toy data:





Group Ranking

Ra	Pickup Group	Reachable	Destination	Medical Need	Severity	Score	Total Route Cost	Travel Time	Danger	Reason
1	Pickup Group 2	☑	Al-Mafraq Hospital	☑	3	57	15	15	0	Reachable with current road
2	Pickup Group 1	☑	Shelter North	☐	2	47	13	13	0	Reachable with current road
3	Pickup Group 4	☑	Shelter West	☐	2	41	15	15	0	Reachable with current road
4	Pickup Group 3	☑	Shelter North	☐	1	29	19	19	0	Reachable with current road

To verify the correctness of our core routing algorithm, we demonstrate how our modified Dijkstra-style algorithm behaves using a simplified example that directly reflects our ***Al-Mafraq Simplified Evacuation Scenario***. This allows us to explicitly trace the decision-making process and confirm that the algorithm correctly computes the safest and most efficient path under our weighted conditions.

To begin the decision-making process, the algorithm first confirms whether the vehicle can reach the pickup. Considering that our toy example is one large connected component, with no closed roads, we can confirm that all pickups are in fact reachable with the current vehicle

location and graph. We can tell that this is true by the reason located on the group ranking page, given that it confirms each pickup is reachable with the current roads and other conditions.

Next, we collect our group of feasible destination nodes. Again, following the same logic as above, since the graph is one large connected component, all destination nodes are reachable. Only the nodes that are valid drop-offs, such as medical needs to the hospital, no medical to shelters, and only to shelters with space, we can tell that for each pickup, they get a list of feasible destination nodes based on their needs. Given the final results, we can tell it is working due to the fact that group 2 is only routed to the hospital due to their need for medical help while being directly connected to a shelter. Of the remaining pick-up groups, each group's possible paths only include ones in which there is enough space, reducing their options and proving functional. For example, pickup group 3 has 14 total occupants, meaning they cannot travel to Shelter West with its remaining capacity of 9. Even though Shelter West is directly connected to group 3's location by edge E8, we know the algorithm functioned properly by the fact that it instead chose to send the group to the north shelter, which has a remaining capacity that exceeds the number of survivors within that group. This step here proves that our decision logic is functioning by making sure the needs of the group are used within decision making, to prevent cases of overflow or drop off not matching the needs.

The last step before ranking our options is to get the best path for each pickup to the destination path. Once again, due to the initial nature of our graph (which can be changed by the user), no routes should be unreachable, which is confirmed in the ranking table. Instead, we can now get the best path for each pickup to the destination, and begin calculating everything. We can see all the calculations presented within the group ranking table after running the planner. Using the priority tuple previously discussed, the program should have placed pickup group 2 on

the way to the hospital as the best choice, given that it satisfies the top 3 requirements of the priority tuple. Not only does this group have medical needs, but they also have the highest severity and score, placing them at the top of the priority list. The second group to be ranked should be group 1, as they have the 4th priority requirement, the best travel cost, and time. Running the simulation shows just that, with pickup group 2 being ranked the best next choice and group 1 being second. Given these situations and conditions, we can verify the algorithm's correctness by the fact that the decision-making algorithm chose the destinations that best fit the needs of the group according to the priority rules implemented.

Conclusion:

While our project was very successful, there are a few things we discovered when developing both the front and backend of the application. One major issue is that the city maps are all custom-made, which, while it can allow users to update any new paths that may appear after any events within the conflict zone, it can make the algorithm unrealistic in the sense that edges don't follow the city streets exactly. While our primary goal was getting our decision-based algorithm and visuals working, one future improvement for our application would be the integration of OpenStreetView and OsmNX, two public Python libraries that can be used to create near-exact graphs based on city layouts. With the time allowed, we decided to leave this out for now, but that is definitely something that could be implemented in a further expansion of this application.

Using the knowledge we gained from this course, within graph theory, shortest path algorithms, and even decision-making algorithms, we were able to combine them all into one large web-based user application to assist in conflict zones. While again, this application does

not seek to solve or end conflicts as a whole, we aimed to try and assist the group most impacted by conflict, the people whose lives are upended in the wake of conflict. Thanks to the guided lectures and homeworks we were able to develop just that, a program that uses rational and logical-based thinking, with a mix of graph and pathfinding algorithms to get citizens the help they need in the best possible order.

AI Usage Disclaimer:

During the completion of this project, AI was consulted and used. AI was initially used to help gather project ideas and determine which topic we were the most interested in diving into. AI was also used to help narrow those ideas and eventually solidify our ideas into a concrete project plan. We used AI to help brainstorm the tools and libraries that could be used to develop this project, and what would work best given our project idea. In the end, we decided to use tools like Networkx, Folium, pandas, etc, which all helped with the project but also kept it simple enough for us to understand how to use and correctly implement these libraries. Lastly, AI was consulted to help us brainstorm and develop parts of the UI design and help understand the Streamlit and folium libraries better, and discover which features we could use to develop a UI that was both visually pleasing and user-friendly.